Smart MCQ Solver Challenge

Milestone 1: Exploratory Data Analysis & Baseline Models

1. Overview

This report documents the first milestone of the Smart MCQ Solver Challenge.
Milestone 1 focused on understanding the dataset and establishing baseline approaches through exploratory data analysis (EDA), text preprocessing, and simple similarity-based ranking.

2. Description

The training dataset (train.csv) consists of MCQ-format questions covering diverse topics, particularly philosophy, science, and reasoning. Each row contains:

- id — unique question identifier
- prompt — the question/problem statement
- A, B, C, D, E — five possible answer choices
- answer — the correct label (only in training data)

The test set (test.csv) has the same structure but without the answer column. A sample_submission.csv demonstrates the format: each row requires exactly three predicted labels separated by spaces (e.g., "B C A").

3. Milestone 1 Questions & Approaches

The following ten questions guided the EDA and baseline modeling work. Each question is described along with the approach taken and the result obtained.

3.1 Detailed Question Walkthroughs

Question 1 — Answer Frequency Distribution

What it asked: Compute how often each answer label (A–E) appears in the training set, identify the most and least frequent labels, and sum those two counts.

Approach: Used pandas value_counts() on the answer column, sorted alphabetically to inspect the distribution.

```
freq = df['answer'].value_counts().sort_index()
most_frequent = freq.max()   # B = 490
least_frequent = freq.min()  # E = 324
total = most_frequent + least_frequent  # 814
```

A: 369   B: 490   C: 459   D: 358   E: 324
Most frequent: B (490) | Least frequent: E (324)

Q1 Answer: 814

Insight: The answer distribution is mildly imbalanced — B and C appear most often, while D and E are underrepresented. This matters for baseline strategies.

Question 2 — Vocabulary Size After Cleaning

What it asked: Determine the total number of unique words across all question prompts after lowercasing and removing punctuation basically after cleaning the data.

Approach: Applied lowercasing and punctuation removal using str.translate(), then exploded the word list and counted unique tokens.

```
df['cleaned'] = df['prompt'].str.lower().str.translate(str.maketrans('', '', string.punctuation))
all_words = df['cleaned'].str.split().explode()
vocab_size = all_words.nunique()  # 859
```

Q2 Answer: 859 unique words

Insight: The vocabulary size of 859 unique words indicates that the prompts use a relatively constrained set of terms.

Question 3 — Stop Word Removal on Row ID 1

What it asked: Take the first question (ID=1), remove standard English stop words, and count the remaining meaningful words.

Approach: Used sklearn's ENGLISH_STOP_WORDS set to filter out common words like 'the', 'is', 'of'. Only content-bearing tokens are retained.

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
filtered = [w for w in words_row1 if w not in ENGLISH_STOP_WORDS]
# Result: 13 words
```

Insight: After removing stop words, the remaining words capture the key concepts of the question. This demonstrates that stop-word removal helps retain meaningful information while eliminating common words that contribute little to understanding the question."

Question 4 — TF-IDF Vocabulary Size

What it asked: Build a TF-IDF vectorizer on the combined text of all prompts and answer options, and tell the exact total number of feature columns (vocabulary size) generated by the vectorizer the vocabulary size.

Approach: Concatenated all six text fields (prompt + A + B + C + D + E) per row, fitted a TfidfVectorizer with built-in English stop word removal.

```
all_texts = []
for _, row in df.iterrows():
    combined = ' '.join([row['prompt'], row['A'], row['B'],
                         row['C'], row['D'], row['E']])
    all_texts.append(combined)
vectorizer = TfidfVectorizer(stop_words='english')
vectorizer.fit(all_texts)
# vocabulary size: 2762
```

Q4 Answer: TF-IDF vocabulary size = 2762

Insight: Including answer options more than triples the vocabulary (from 859 to 2762), reflecting that answer options introduce additional domain terminology.

Question 5 — Cosine Similarity

What it asked: For the first question, compute the cosine similarity between the TF-IDF vector of the prompt and the TF-IDF vector of option A.

Approach: Used the already-fitted TF-IDF vectorizer to transform individual text fields, then applied sklearn's cosine_similarity function.

```
prompt_vec = vectorizer.transform([row1['prompt']])
optionA_vec = vectorizer.transform([row1['A']])
sim = cosine_similarity(prompt_vec, optionA_vec)[0][0]
# sim = 0.272
```

Q5 Answer: Cosine similarity = 0.272

Insight: A cosine similarity score of 0.272 indicates some lexical overlap between the prompt and option A, though the overlap is not particularly strong.

Question 6 — Accuracy of Greedy Cosine Similarity Selection

What it asked: Evaluate a simple strategy: for each question, predict the answer option with the highest cosine similarity to the prompt. What is the accuracy?

Approach: Looped through every row, computed cosine similarities for all five options, selected the one with the maximum score, and checked against the true label.

```
for _, row in df.iterrows():
  prompt_vec = vectorizer.transform([str(row['prompt'])])
  sims = {}
  for opt in ['A', 'B', 'C', 'D', 'E']:
      opt_vec = vectorizer.transform([str(row[opt])])
```

```
        sims[opt] = cosine_similarity(prompt_vec, opt_vec)[0][0]
    predicted = max(sims, key=sims.get)
    if predicted == row['answer']:
        correct += 1
```

Q6 Answer: Accuracy = 13.55%

Insight: The TF-IDF cosine similarity approach achieved only 13.55% accuracy, which is lower than the 20% accuracy expected from uniform random guessing. This suggests that lexical overlap between the prompt and correct answer is often misleading in this dataset.

Questions 7 & 8 — Understanding MAP@3

What they asked: These two questions tested understanding of the MAP@3 evaluation metric through concrete examples.
MAP@3 rewards finding the correct answer earlier in a ranked list of 3 predictions. The formula returns 1/(rank) when the correct answer is found, or 0 if it is not in the top 3.

Q	Prediction	True Answer	MAP@3 Score
Q7	[C, A, B]	C	1.0 (rank 1)
Q8	[D, B, E]	B	0.5 (rank 2)

Insight: MAP@3 is a forgiving metric — even placing the correct answer second still earns 50% of the maximum score. This makes smart ranking more valuable than binary correctness

Question 9 — Majority Class Baseline MAP@3

What it asked: Build the simplest possible baseline: always predict the three most frequent answer labels in the training set, in frequency order. What is the MAP@3?

Approach: Computed label frequencies, extracted the top 3 (B, C, A), and applied map_at_3 against every row's true answer.

```
top3 = freq.index[:3].tolist()  # ['B', 'C', 'A']
scores = [map_at_3(row['answer'], top3) for _, row in df.iterrows()]
MAP@3 = sum(scores) / len(scores)  # 0.4213
```

Top 3 most frequent: ['B', 'C', 'A']
Q9 Answer: Majority Class Baseline MAP@3 = 0.4213

Insight: The majority-class baseline achieves a MAP@3 score of 0.4213 despite using no textual information. This highlights the impact of class imbalance and establishes a surprisingly competitive baseline

Question 10 — TF-IDF Pipeline MAP@3
What it asked: Build a model that ranks answer options using TF-IDF similarity, take top 3 guesses, and measure how well this ranking matches the correct answers across the entire dataset..

Approach: For each row, computed cosine similarities between the prompt vector and all five option vectors, then sorted options in descending order of similarity to produce a ranked top-3 prediction.

```
for _, row in df.iterrows():
    sims = {}
    for opt in ['A','B','C','D','E']:
        opt_vec = vectorizer.transform([str(row[opt])])
        sims[opt] = cosine_similarity(prompt_vec, opt_vec)[0][0]
        prompt_vec = vectorizer.transform([str(row['prompt'])])
    top3_pred = sorted(sims, key=sims.get, reverse=True)[:3]
    scores_tfidf.append(map_at_3(row['answer'], top3_pred))
```

Q10 Answer: TF-IDF Pipeline MAP@3 = 0.2962

Insight: The TF-IDF pipeline (0.2962) underperforms the naive majority baseline (0.4213). Lexical overlap between a question and its correct answer is simply not a reliable signal for this type of reasoning-based MCQ dataset.

4. Key Findings & Analysis

4.1 Baseline Comparison

Approach	Score	Notes
Random Guess (uniform)	~0.20 (Acc)	Lower bound
Greedy Cosine	13.55% Acc	Below random
TF-IDF Pipeline MAP@3	0.2962	Lexical only
Majority Class Baseline	0.4213	Better naive baseline

4.2 Why TF-IDF Fails Here

These questions require genuine reasoning — understanding philosophical positions, scientific principles, or logical implications. TF-IDF similarity measures only vocabulary overlap, not meaning. A correct answer to a question may share no key words with the prompt while still being semantically correct.

4.3 Why the Majority Baseline Is Hard to Beat

Because the dataset has mild class imbalance (B=490, C=459, A=369 together = 1318 out of 2000 rows, ~66%), always predicting [B, C, A] guarantees coverage of the correct answer two-thirds of the time. Any semantic model must provide genuine discriminative power to surpass 0.4213 MAP@3.

Limitations

- TF-IDF captures only word frequency and overlap.
- It does not understand semantic meaning.
- Philosophy and reasoning questions often require contextual understanding.
- Similar wording does not necessarily indicate the correct answer.

Conclusion

It successfully established the data landscape and quantified two baseline approaches. The key takeaway is that surface-level lexical matching (TF-IDF cosine similarity) is insufficient for reasoning-based MCQ tasks, while the naive majority baseline sets a surprisingly competitive bar due to class imbalance.

The path forward requires semantic understanding — either through pre-trained language models or LLM-based prompting strategies — to achieve meaningful improvements over these baselines.